

RAPPORT TECHNIQUE

GreenFinance-Scorer

Étape 1 — Préparation de l'environnement
Trace technique complète des sous-tâches 1.1 à 1.8

Projet	GreenFinance-Scorer — plateforme de scoring ESG et carbone
Dépôt	<code>github.com/cheikhmama/GreenFinance-Scorer</code>
Étape documentée	Étape 1 / 21 — Préparation de l'environnement (Phase 0)
Sous-tâches	8 (Prompts 1.1 → 1.8), toutes validées
Date du rapport	11 août 2026
Commits associés	82e9e2b, 5b777ec (branche main)
Statut final	TOUS CRITÈRES VALIDÉS

1. Résumé exécutif

Ce document trace intégralement l'exécution de l'Étape 1 du plan d'implémentation GreenFinance-Scorer : la préparation de l'environnement de développement backend, décomposée en 8 sous-tâches (Prompts 1.1 à 1.8), chacune suivant le cycle *Analyse* → *Préparation* → *Implémentation* → *Test* → *Vérification* → *Validation*.

Les 8 sous-tâches ont été menées à terme avec succès. Trois incidents techniques significatifs sont survenus en cours de route — un dépôt git mal scopé à la racine du répertoire utilisateur, des interruptions réseau répétées lors du téléchargement d'images Docker, et un conflit de port 5432 avec une instance PostgreSQL native Windows préexistante. Chacun est documenté avec son diagnostic, sa correction et sa vérification.

Résultat des tests	4/4 tests passés (unitaires + intégration), couverture 89 %
Qualité du code	<code>ruff check .</code> et <code>mypy app/</code> : aucune erreur
Infrastructure	PostgreSQL 16 + pgvector, Redis 7, API FastAPI — 3 conteneurs opérationnels
Migrations	Alembic initialisé, réversibilité confirmée (upgrade/downgrade/upgrade)
CI	Workflow GitHub Actions créé et validé étape par étape en local
Sécurité	Aucun secret en dur ni commité ; <code>.env</code> correctement ignoré

Table des matières

1. Résumé exécutif
2. Anomalie préliminaire — dépôt git mal scopé
3. Sous-tâche 1.1 — Structure de dossiers
4. Sous-tâche 1.2 — Gestion des dépendances
5. Sous-tâche 1.3 — Docker Compose (db + redis)
6. Sous-tâche 1.4 — Variables d'environnement
7. Sous-tâche 1.5 — API FastAPI minimale + Dockerfile
8. Sous-tâche 1.6 — Connexion base de données + `/health` réel
9. Sous-tâche 1.7 — Initialisation Alembic
10. Sous-tâche 1.8 — Intégration continue (CI)
11. Incident majeur — Conflit de port 5432 et résolution
12. Intégration Git / GitHub
13. Bilan final — critères de passage à l'Étape 2
14. Annexes — arborescence, dépendances, variables d'environnement

2. Anomalie préliminaire — dépôt git mal scopé

Détectée avant le démarrage du Prompt 1.1

L'inspection initiale du dossier `Green-Finance-Scorer` a révélé que le dépôt git actif n'était pas local au projet : `git rev-parse --show-toplevel` pointait vers `C:\Users\hpi7`, la racine du répertoire utilisateur entier.

CONSTAT

Dépôt `.git` de 781 Mo à la racine du home, 28 commits, remote configuré vers `github.com/houssam-karouane/mlops-financial-pipeline.git` — un projet tiers sans rapport avec `GreenFinance-Scorer`. Aucun fichier sous `Green-Finance-Scorer/` n'y était suivi. Toute commande `git status` ou `git add` lancée depuis le dossier du projet aurait remonté des milliers de fichiers personnels sans rapport (notebooks, configurations, historiques divers).

DÉCISION

Validée explicitement par l'utilisateur : création d'un dépôt git dédié, isolé, directement dans `Green-Finance-Scorer/`, sans modifier ni toucher au dépôt existant à la racine du home.

```
$ git init
Initialized empty Git repository in C:/Users/hpi7/OneDrive/Documents/Green-Finance-Scorer/.git/
```

3. Sous-tâche 1.1 — Initialisation du dépôt et structure de dossiers

OBJECTIF

Poser la structure de dossiers de référence du projet backend, sans aucune logique métier.

CONFIGURATION MISE EN PLACE

- ✓ 18 dossiers créés : `app/core`, `app/auth`, `app/ingestion`, `app/scoring`, `app/carbon`, `app/explainability`, `app/admin`, `app/audit`, `app/company`, `app/investor`, `app/researcher`, `app/institution`, `tests/unit`, `tests/integration`, `config/weights`, `config/carbon`, `notebooks`, `storage`
- ✓ 16 fichiers `__init__.py` vides (14 packages domaine + `app` + `tests`)
- ✓ `.gitignore` Python standard (`venv`, `__pycache__`, `.env`, `storage/*` sauf `storage/.gitkeep`)
- ✓ `README.md` — titre et description uniquement

COMMANDES EXÉCUTÉES

```
mkdir -p app/core app/auth app/ingestion app/scoring app/carbon app/explainability \
      app/admin app/audit app/company app/investor app/researcher app/institution \
      tests/unit tests/integration config/weights config/carbon notebooks storage

for d in app app/core app/auth ... tests tests/unit tests/integration; do : >
"$d/__init__.py"; done
touch storage/.gitkeep
```

RÉSULTATS OBTENUS

```
≡ nombre de __init__.py ≡
16
≡ git status ≡
On branch main
No commits yet
Untracked files:
.gitignore README.md app/ storage/ tests/
```

ERREURS RENCONTRÉES

Aucune, hors l'anomalie de dépôt git traitée en section 2.

CRITÈRES DE VALIDATION

- ✓ Structure strictement conforme à la liste demandée, aucun dossier manquant
- ✓ `git status` correctement scopé au projet (plus de pollution du répertoire home)

4. Sous-tâche 1.2 — Gestion des dépendances

OBJECTIF

Dépendances de base installables et verrouillées via uv.

CONFIGURATION MISE EN PLACE

pyproject.toml — Python ≥ 3.11 :

Dépendances principales	fastapi, uvicorn[standard], sqlalchemy, alembic, psycopg2-binary, pydantic-settings, pytest, pytest-cov
Groupe dev	ruff, mypy, pre-commit

COMMANDES EXÉCUTÉES

```
python -m pip install --user uv          # uv absent, installation via pip (v0.12.3)
uv sync                                # installation + génération de uv.lock
```

CORRECTION MINEURE

Ni uv ni poetry n'étaient présents sur la machine. Conformément à la consigne (uv en priorité, poetry en repli), uv a été installé via `pip install --user uv`. L'exécutable a nécessité l'ajout explicite de `AppData\Roaming\Python\Python312\Scripts` au PATH de chaque commande (l'état du shell ne persiste pas entre les appels de l'outil Bash).

RÉSULTATS OBTENUS

```
Prepared 52 packages in 3m 00s
Installed 52 packages in 2.43s
+ fastapi==0.141.1 + sqlalchemy==0.0.39 + alembic==1.19.1 + psycopg2==3.3.4 ...
$ uv run python -c "import fastapi, sqlalchemy, alembic; print('OK', ...)"
OK 0.141.1 0.0.39 1.19.1
```

CRITÈRES DE VALIDATION

- ✓ Import des trois modules sans erreur
- ✓ uv.lock généré (353 Ko)
- ✓ Installation propre sur environnement, aucun conflit de version

5. Sous-tâche 1.3 — Docker Compose (base de données et file de tâches)

OBJECTIF

PostgreSQL (avec pgvector) et Redis démarrables et accessibles, sans service applicatif à ce stade.

CONFIGURATION MISE EN PLACE

```
services:
  db:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_USER: greenfinance
      POSTGRES_PASSWORD: changeme
      POSTGRES_DB: greenfinance
    volumes: [pgdata:/var/lib/postgresql/data]
    ports: ["5432:5432"]
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
volumes:
  pgdata:
```

ERREURS RENCONTRÉES ET CORRECTIONS

ERREUR 1 — DAEMON DOCKER NON PRÊT

```
unable to get image 'redis:7-alpine': failed to connect to the docker API at
npipe://.../pipe/dockerDesktopLinuxEngine ... The system cannot find the file specified.
```

Docker Desktop était en cours de démarrage au moment de la première tentative.

CORRECTION

Nouvelle tentative après quelques instants — `docker info` confirmait le daemon prêt (Name: `docker-desktop`).

ERREUR 2 & 3 — INTERRUPTIONS RÉSEAU PENDANT LE PULL

```
short read: expected 111767025 bytes but got 67782892: unexpected EOF
...
short read: expected 12414037 bytes but got 2975064: unexpected EOF
```

Deux échecs consécutifs du téléchargement de l'image `pgvector/pgvector:pg16` (~112 Mo), sur des calques différents à chaque fois — instabilité réseau/proxy Docker Desktop, pas un problème de configuration.

CORRECTION

Troisième tentative de `docker compose pull db redis` : succès complet.

COMMANDES EXÉCUTÉES ET RÉSULTATS

```
$ docker compose up -d db redis
$ docker compose ps
NAME                                STATUS              PORTS
green-finance-scorer-db-1          Up 12 seconds      0.0.0.0:5432→5432/tcp
green-finance-scorer-redis-1       Up 12 seconds      0.0.0.0:6379→6379/tcp

$ docker compose exec db psql -U greenfinance -c "SELECT 1;"
?column?
-----
1

$ docker compose exec redis redis-cli ping
PONG

$ docker compose down && docker compose up -d db redis    # test de redémarrage propre
# → services recréés, volume pgdata préservé, psql et redis-cli à nouveau OK
```

CRITÈRES DE VALIDATION

- ✓ Les deux services démarrent et répondent (« healthy »/« running »)
- ✓ SELECT 1 et PING répondent correctement
- ✓ Redémarrage propre après down && up confirmé

6. Sous-tâche 1.4 — Variables d'environnement

OBJECTIF

Configuration centralisée, aucune valeur en dur dans le code.

CONFIGURATION MISE EN PLACE

`.env.example` (valeurs d'exemple, aucune vraie clé) :

```
DATABASE_URL, REDIS_URL, SECRET_KEY, MFA_ISSUER_NAME, ANTHROPIC_API_KEY,  
STORAGE_BACKEND, STORAGE_PATH, DEFAULT_SCORING_CONFIG, EMISSION_FACTORS_PATH, ENVIRONMENT
```

`app/core/config.py` — classe `Settings` (`pydantic-settings`) : `DATABASE_URL` et `REDIS_URL` disposent de valeurs par défaut raisonnables pour le développement local ; les 8 autres variables n'ont aucune valeur par défaut et sont donc obligatoires.

COMMANDES EXÉCUTÉES ET RÉSULTATS

```
$ cp .env.example .env # copie locale pour les tests (fichier ignoré par git)
$ uv run python -c "from app.core.config import Settings; print(Settings())"
database_url=... secret_key='changeme-dev-secret-key' ... environment='development' # OK

# Test du cas d'erreur : .env retiré temporairement, aucune variable d'environnement définie
$ uv run python -c "from app.core.config import Settings; Settings()"
pydantic_core._pydantic_core.ValidationError: 8 validation errors for Settings
secret_key          Field required [type=missing]
mfa_issuer_name     Field required [type=missing]
anthropic_api_key   Field required [type=missing]
storage_backend     Field required [type=missing]
storage_path        Field required [type=missing]
default_scoring_config Field required [type=missing]
emission_factors_path Field required [type=missing]
environment         Field required [type=missing]
```

CRITÈRES DE VALIDATION

- ✓ Chargement réussi avec `.env` complet
- ✓ Erreur explicite et détaillée (8 champs listés) si des variables obligatoires manquent — pas d'échec silencieux
- ✓ `.env` absent de `git status` ; `.env.example` bien suivi

7. Sous-tâche 1.5 — API FastAPI minimale avec endpoint de santé

OBJECTIF

Premier point d'entrée applicatif, testable en local et en conteneur.

CONFIGURATION MISE EN PLACE

- ✓ `app/main.py` — FastAPI (titre/version issus de `Settings`), `GET /health`
- ✓ `Dockerfile` — `python:3.11-slim`, installation de `uv`, `uv sync`
- ✓ Service `api` ajouté à `docker-compose.yml` (port 8000, `env_file: .env`, `depends_on: db, redis`)
- ✓ `tests/unit/test_health.py` (FastAPI `TestClient`)

ERREUR — DÉPENDANCE MANQUANTE

`TestClient` nécessite `httpx`, non couvert par la liste de dépendances du Prompt 1.2 (`fastapi` seul ne l'inclut pas).

```
ModuleNotFoundError: No module named 'httpx'
```

CORRECTION

Ajout de `httpx` au groupe dev de `pyproject.toml` (dépendance de test uniquement), puis `uv sync`.

AJUSTEMENT — PARITÉ LOCAL/CONTENEUR

Le `Dockerfile` excluait initialement les dépendances dev (`--no-dev`), empêchant l'exécution de `pytest` dans le conteneur. Le critère de validation exige que le test passe identiquement en local et en conteneur : les dépendances dev et le dossier `tests/` ont donc été inclus dans l'image.

RÉSULTATS OBTENUS

```
$ uv run pytest tests/unit/test_health.py -v
tests/unit/test_health.py::test_health_returns_ok PASSED

$ docker compose up -d --build api
$ curl http://localhost:8000/health
{"status":"ok"} # HTTP 200

$ docker compose exec api pytest tests/unit/test_health.py -v
tests/unit/test_health.py::test_health_returns_ok PASSED # identique au run local
```

CRITÈRES DE VALIDATION

- ✓ `pytest` passe en local
- ✓ `/health` répond 200 via `curl` sur le conteneur
- ✓ `pytest` passe à l'identique dans le conteneur Docker

8. Sous-tâche 1.6 — Connexion à la base de données via SQLAlchemy

OBJECTIF

Connexion base de données vérifiée au démarrage et à chaque requête — pas seulement supposée.

CONFIGURATION MISE EN PLACE

- ✓ `app/core/database.py` — engine SQLAlchemy, `get_session()` (dépendance FastAPI), exception `DatabaseConnectionError`, `check_database_connection()` (SELECT 1)
- ✓ `app/main.py` — `lifespan` appelle la vérification au démarrage (log d'erreur sans crash) ; `/health` vérifie la connexion à chaque requête et retourne `{"status": "ok", "database": "connected"}` (200) ou `{"status": "error", "database": "unreachable", ...}` (503)
- ✓ `tests/integration/test_database.py` — 3 tests (connexion OK, dégradation sans crash via mock, exception explicite sur moteur invalide)

INCIDENT MAJEUR — ÉCHEC D'AUTHENTIFICATION PASSWORD

```
psycopg.OperationalError: connection failed: ... FATAL: password authentication failed for user "greenfinance"
```

Diagnostic complet documenté en **section 11** (conflit de port 5432 avec une instance PostgreSQL native Windows préexistante). Corrigé temporairement par un remap de port (5433) le temps de l'Étape 1, puis résolu définitivement après coup — voir section 11.

BUG LATENT DÉCOUVERT — RÉOLUTION RÉSEAU INTERNE DOCKER

À l'intérieur du conteneur `api`, `DATABASE_URL` utilisait `localhost`, qui à l'intérieur du réseau Docker désigne le conteneur `api` lui-même, pas le service `db`.

CORRECTION

Ajout d'un override `environment` : pour le service `api` dans `docker-compose.yml`, pointant explicitement vers `db:5432` et `redis:6379` (noms de service, résolution DNS interne Docker).

CORRECTION — TEST LENT

Le test simulant une base injoignable via `localhost:1` prenait plus de 4 minutes (timeout TCP par défaut). Ajout de `connect_args={"connect_timeout": 2}` — échec ramené à ~2 secondes.

RÉSULTATS OBTENUS (APRÈS CORRECTIONS)

```
$ curl http://localhost:8000/health
{"status":"ok","database":"connected"} # HTTP 200

$ docker compose stop db && curl http://localhost:8000/health
{"status":"error","database":"unreachable","detail":"..."} # HTTP 503, pas de crash serveur

$ docker compose start db && curl http://localhost:8000/health
{"status":"ok","database":"connected"} # HTTP 200, rétabli automatiquement

$ uv run pytest tests/ -v
4 passed in 5.23s
```

CRITÈRES DE VALIDATION

- ✓ /health reflète l'état réel de la connexion (pas de « ok » inconditionnel)
- ✓ Base arrêtée → erreur explicite, serveur non planté
- ✓ Base redémarrée → /health rétabli sans redémarrage de l'API

9. Sous-tâche 1.7 — Initialisation d'Alembic

OBJECTIF

Migrations de schéma versionnées et opérationnelles, sans modèle métier encore défini.

CONFIGURATION MISE EN PLACE

- ✓ `alembic init alembic`
- ✓ `alembic/env.py` modifié : `config.set_main_option("sqlalchemy.url", get_settings().database_url)` — source unique : `app.core.config.Settings`
- ✓ `alembic.ini` : ligne `sqlalchemy.url` retirée (remplacée par un commentaire explicatif) pour qu'aucune valeur en dur ne subsiste
- ✓ Première migration vide générée : `440af0be0e05_initial_empty_migration.py`

COMMANDES EXÉCUTÉES ET RÉSULTATS

```
$ alembic upgrade head
Running upgrade → 440af0be0e05, initial empty migration
$ alembic downgrade base
Running downgrade 440af0be0e05 → , initial empty migration
$ alembic upgrade head
Running upgrade → 440af0be0e05, initial empty migration
$ alembic current
440af0be0e05 (head)
```

CRITÈRES DE VALIDATION

- ✓ `upgrade head` sur base vide sans erreur
- ✓ Réversibilité confirmée (`downgrade base` puis `upgrade head`)
- ✓ Preuve structurelle : `alembic.ini` ne contient plus de `sqlalchemy.url` — la valeur provient exclusivement de `Settings`/l'environnement

10. Sous-tâche 1.8 — Intégration continue minimale

OBJECTIF

Chaque changement futur vérifié automatiquement avant fusion.

ERREURS DE BASELINE DÉTECTÉES AVANT LA MISE EN PLACE DE LA CI

`ruff check .` : 10 erreurs dans les fichiers générés par `alembic init` (imports non triés, `typing.Union` déprécié, imports inutilisés, `pass` superflus).
`mypy app/` : 8 faux positifs sur l'appel `Settings()` (le plugin `mypy` de `pydantic` n'était pas configuré).

CORRECTIONS

`ruff check . --fix` (11 corrections automatiques) ; ajout de `[tool.mypy] plugins = ["pydantic.mypy"]` dans `pyproject.toml`. Les deux outils passent ensuite sans erreur.

CONFIGURATION MISE EN PLACE

`.github/workflows/ci.yml` — déclenché sur `push/pull_request`, étapes séquentielles (arrêt à la première échec par défaut) :

(boucle 30×1s)

#	Étape	Commande
1	Installation des dépendances	<code>uv sync</code>
2	Lint	<code>uv run ruff check .</code>
3	Typage	<code>uv run mypy app/</code>
4	Services	<code>docker compose up -d db redis</code>
5	Attente DB prête	<code>pg_isready</code>
6	Environnement	<code>cp .env.example .env</code>
7	Tests + couverture	<code>uv run pytest --cov=app --cov-report=term-missing</code>

LIMITE DE VALIDATION RECONNUE

Aucun remote GitHub n'existait au moment de ce prompt — impossible de pousser une branche de test et d'observer le workflow réellement s'exécuter dans le cloud GitHub Actions.

VALIDATION ALTERNATIVE EFFECTUÉE

Rejeu manuel de chaque étape du workflow, dans l'ordre exact, en local : succès complet de bout en bout. Puis introduction volontaire d'une erreur de lint (`import os` inutilisé) :

```
$ uv run ruff check .  
F401 `os` imported but unused  
Found 1 error.  
EXIT CODE: 1
```

confirmant l'échec immédiat et donc l'arrêt de la chaîne avant les étapes suivantes (comportement par défaut de GitHub Actions pour des étapes séquentielles sans `continue-on-error`). L'erreur a ensuite été retirée.

CRITÈRES DE VALIDATION

- ✓ `ruff` et `mypy` propres
- ✓ Chaîne complète rejouée localement avec succès (7 étapes)
- ✓ Comportement fail-fast démontré (erreur de lint volontaire → arrêt immédiat, code sortie 1)
- ✓ Badge CI ajouté au `README.md`

11. Incident majeur transverse — Conflit de port 5432

11.1 — Diagnostic

Découvert pendant la sous-tâche 1.6, lors de l'échec d'authentification PostgreSQL depuis l'hôte.

```
$ netstat -ano | grep ":5432"
TCP    0.0.0.0:5432    LISTENING    9596
TCP    0.0.0.0:5432    LISTENING    9436

$ Get-Process -Id 9436,9596 | Select Id, ProcessName, Path
9596  com.docker.backend  C:\Program Files\Docker\Docker\resources\com.docker.backend.exe
9436  postgres
```

CAUSE RACINE

Une instance **PostgreSQL 18 native Windows** (service postgresql-x64-18, démarrage Automatic), installée sur la machine indépendamment de ce projet, écoutait déjà sur le port 5432 et interceptait les connexions destinées au conteneur Docker. La vérification via `docker compose exec db psql` ne détectait pas le problème car elle passe par le socket Unix interne du conteneur (authentification `trust`), pas par le port TCP exposé.

Élément	Valeur
Service Windows	postgresql-x64-18
Binaires	C:\Program Files\PostgreSQL\18\bin\
Répertoire de données	C:\Program Files\PostgreSQL\18\data
Démarrage d'origine	Automatic

11.2 — Correctif temporaire (pendant l'Étape 1)

Remap du port hôte de db vers 5433 (port interne au réseau Docker inchangé, toujours 5432) ; mise à jour de `.env`, `.env.example` et de la valeur par défaut dans `config.py`. Choisi pour ne pas toucher à un service Windows préexistant sans validation explicite de l'utilisateur.

11.3 — Résolution définitive (après l'Étape 1, sur demande explicite)

L'utilisateur a demandé l'arrêt et la désactivation permanente du service natif afin de revenir au port standard 5432 prévu par le plan.

BLOPAGE — DROITS ADMINISTRATEUR REQUIS

```
Stop-Service : Cannot open postgresql-x64-18 service on computer '.'.
Set-Service  : Access is denied
```

La session PowerShell utilisée par l'assistant n'étant pas élevée (et l'élévation UAC interactive n'étant pas possible dans cet environnement), les commandes ont été fournies à l'utilisateur pour exécution dans un PowerShell administrateur.

RÉSOLU PAR L'UTILISATEUR

```
PS (Admin)> Stop-Service -Name "postgresql-x64-18" -Force
PS (Admin)> Set-Service -Name "postgresql-x64-18" -StartupType Disabled
PS (Admin)> Get-Service -Name "postgresql-x64-18" | Select Name, Status, StartType
Name                Status  StartType
postgresql-x64-18  Stopped Disabled

PS (Admin)> Get-NetTCPConnection -LocalPort 5432 -State Listen
No matching MSFT_NetTCPConnection objects found          # port libre
```

Suite à la libération du port, la configuration du projet a été repassée sur 5432 (`docker-compose.yml`, `.env`, `.env.example`, `config.py`), les conteneurs recréés, et l'intégralité de la suite de vérifications rejouée avec succès :

```
$ docker compose ps
db      Up    0.0.0.0:5432→5432/tcp
redis   Up    0.0.0.0:6379→6379/tcp
api     Up    0.0.0.0:8000→8000/tcp

$ curl http://localhost:8000/health      → {"status":"ok","database":"connected"} HTTP
200
$ uv run pytest tests/ -v --cov=app      → 4 passed, couverture 89%
$ uv run alembic downgrade base / upgrade head → OK, réversible
$ uv run ruff check . / uv run mypy app/ → propres
```

CRITÈRES DE VALIDATION

- ✓ Port 5432 libéré durablement (survit à un redémarrage Windows, service désactivé)
- ✓ Configuration projet alignée sur le port standard du plan d'origine
- ✓ Suite complète revalidée après le changement (aucune régression)

12. Intégration Git / GitHub

— feat: initialiser l'environnement du projet (Étape 1) — 36 fichiers — fix: revenir au port standard 5432 pour PostgreSQL — 3 fichiers

Élément	Détail
Dépôt créé	Dédié, local à <code>Green-Finance-Scorer/</code> (voir section 2)
Remote	<code>origin</code> → <code>https://github.com/cheikhmama/GreenFinance-Scorer.git</code>
Badge CI	Mis à jour dans <code>README.md</code> avec le chemin réel du dépôt
Commit 1	<code>82e9e2b</code>
Commit 2	<code>5b777ec</code>
Push	EN ATTENTE — confirmation utilisateur requise avant tout <code>git push</code>

HYGIÈNE GIT

`.claude/` (état interne de l'outil, ex. `scheduled_tasks.lock`) ajouté au `.gitignore` avant le premier commit pour éviter de versionner des artefacts d'outillage. Vérification par `git add -A --dry-run` avant chaque commit : aucun secret, aucun fichier inattendu.

13. Bilan final — critères de passage à l'Étape 2

Critère (fin d'Étape 1)	Statut
Les 8 sous-tâches exécutées, tests individuels passés	VALIDÉ
<code>docker compose up -d</code> démarre db + redis + api sans erreur	VALIDÉ
<code>pytest</code> passe intégralement en local	VALIDÉ (4/4, couverture 89 %)
<code>pytest</code> passe intégralement en CI	VALIDÉ PAR REJEU LOCAL (pas de remote au moment du test)
<code>alembic upgrade head</code> sur base vide, réversible	VALIDÉ
Aucune valeur sensible en dur ou commitée	VALIDÉ

Tous les critères de passage à l'**Étape 2 — Architecture du projet** sont remplis. Le plan détaillé de l'Étape 2 (Prompts 2.1 à 2.6) a été présenté séparément pour validation avant implémentation.

14. Annexes

14.1 — Arborescence finale du dépôt

```
Green-Finance-Scorer/
├── .github/workflows/ci.yml
├── .env.example          (suivi)      .env  (ignoré)
├── .gitignore
├── Dockerfile
├── README.md
├── alembic.ini
├── alembic/
│   ├── env.py
│   ├── script.py.mako
│   └── versions/440af0be0e05_initial_empty_migration.py
├── app/
│   ├── main.py
│   ├── core/{config.py, database.py, __init__.py}
│   └── {admin, audit, auth, carbon, company, explainability,
│       ingestion, institution, investor, researcher, scoring}/__init__.py
├── config/{weights, carbon}/      (vides, réservés étapes 12-13)
├── docker-compose.yml
├── notebooks/                    (vide, réservé)
├── pyproject.toml
├── storage/.gitkeep
├── tests/
│   ├── unit/test_health.py
│   └── integration/test_database.py
└── uv.lock
```

14.2 — Dépendances verrouillées (extrait significatif)

Paquet	Version	Paquet	Version
fastapi	0.141.1	sqlmodel	0.0.39
alembic	1.19.1	psycopg	3.3.4
sqlalchemy	2.0.51	pydantic	2.13.4
pydantic-settings	2.15.0	uvicorn	0.52.1
pytest	9.1.1	pytest-cov	7.1.0
ruff	0.16.2	mypy	2.3.0
httpx	0.28.1	pre-commit	4.6.2

14.3 — Variables d'environnement (noms uniquement)

Variable	Valeur par défaut ?	Usage
DATABASE_URL	Oui (dev, localhost:5432)	Connexion PostgreSQL

REDIS_URL	Oui (dev, localhost:6379)	Connexion Redis
SECRET_KEY	Non — obligatoire	Signature/sécurité applicative
MFA_ISSUER_NAME	Non — obligatoire	Authentification à deux facteurs (étape 9)
ANTHROPIC_API_KEY	Non — obligatoire	Intégration Claude (étape 6)
STORAGE_BACKEND	Non — obligatoire	Type de stockage documentaire
STORAGE_PATH	Non — obligatoire	Emplacement de stockage local
DEFAULT_SCORING_CONFIG	Non — obligatoire	Configuration scoring ESG (étape 12)
EMISSION_FACTORS_PATH	Non — obligatoire	Facteurs d'émission carbone (étape 13)
ENVIRONMENT	Non — obligatoire	Mode d'exécution (development/production)